

Schema Evolution on S3-Compatible Object Storage

Zied Ellouzi, Nassim Maliki

1. Introduction

Modern analytical systems continuously evolve over time. In practice, datasets rarely keep the exact same structure forever, and new columns are frequently introduced as applications evolve. One important challenge is ensuring that older datasets remain queryable after schema modifications. This problem becomes particularly important in object-storage environments where multiple dataset versions may coexist simultaneously.

The objective of this project was implementing and benchmarking a simple schema evolution workflow on S3-compatible object storage using MinIO and Parquet datasets.

The project focuses on three main objectives:

- generating versioned datasets;
- preserving backward compatibility between schema versions;
- evaluating the performance impact of mixed version reads.

The implementation combines Parquet datasets, MinIO object storage, schema contracts, automated benchmarks, and backward-compatible reads.

2. Schema Evolution Scenario

The project simulates a simple schema evolution scenario using two dataset versions.

The initial schema (v1) contains the following columns:

- ts
- user_id
- region
- event_type
- value

The second schema version (v2) introduces a new nullable column called device_type.

Column	v1	v2
ts	Yes	Yes
user_id	Yes	Yes
region	Yes	Yes
event_type	Yes	Yes
value	Yes	Yes
device_type	No	Yes

This evolution simulates a realistic situation where new analytical information becomes available over time while older datasets still need to remain compatible. The main challenge of the project is therefore ensuring that both schema versions remain queryable together.

3. Project Architecture

The implementation is divided into several Python components responsible for dataset generation, object-storage transfers, schema alignment, and benchmark execution. `dataset_gen.py` generates synthetic datasets, `upload.py` and `download.py` handle MinIO transfers, `reader_s3.py` aligns schema versions during reads, while `bench.py` executes the complete benchmark workflow. The notebook `analysis.ipynb` is used for result analysis and visualization.

The datasets are stored inside the `ccbd-data` MinIO bucket using separate prefixes for each schema version.

```
ccbd-data/  
├── curated/  
│   ├── events/  
│   │   ├── v1/  
│   │   └── v2/  
├── published/  
│   ├── events/  
│   └── schema.json
```

The `schema.json` file defines the unified schema contract used during validation.

4. Synthetic Dataset Generation

The project uses synthetic datasets generated with non-uniform distributions. The objective was not generating fully realistic business data but creating a more representative analytical workload than purely random uniform data.

The generated event distribution follows a simple funnel-like structure where view events remain dominant, while click and purchase events become progressively less frequent. The evolved schema in v2 also introduces the `device_type` column, which is intentionally dominated by mobile traffic.

These distributions make the benchmark slightly closer to a realistic analytical pipeline while keeping the implementation simple.

Figure 1 and Figure 2 illustrate the generated dataset distributions.

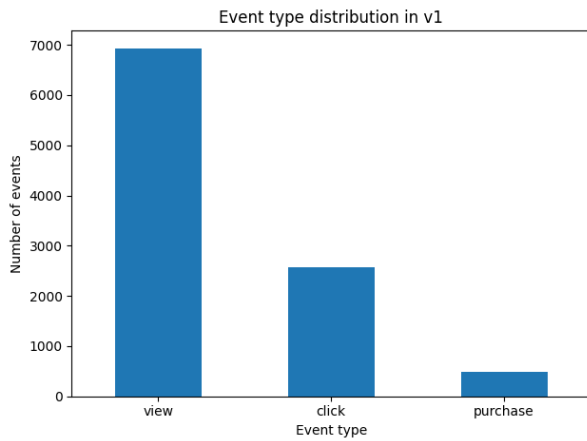


Figure 1. Event type distribution in v1

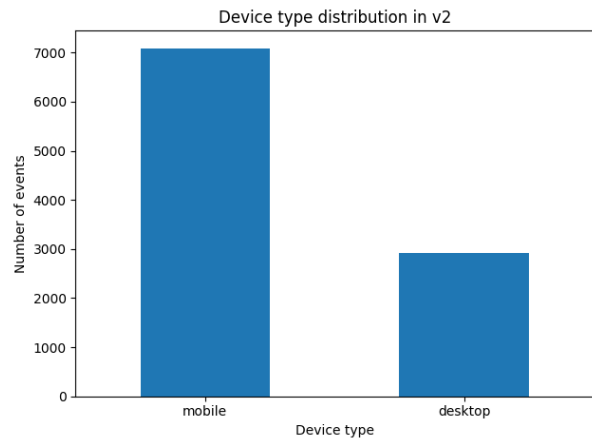


Figure 2. Device type distribution in v2

5. Benchmark Workflow

Once the datasets are generated, the benchmark executes the complete workflow automatically. The workflow generates datasets, clears previous objects from MinIO, uploads datasets to object storage, downloads datasets locally, executes analytical reads, and stores benchmark results.

The benchmark evaluates three reading modes:

- v1, which reads the original schema only;
- v2, which reads the evolved schema only;
- mixed, which reads both schema versions together.

The mixed mode represents the central schema evolution scenario because it combines multiple schema versions into a unified analytical view before query execution.

6. Dataset Sizes

The benchmark was executed using three dataset sizes:

- S (10,000 rows per version)
- M (100,000 rows per version)
- L (1,000,000 rows per version)

The mixed mode combines both versions during query execution.

The objective was not measuring production cloud performance but validating that the workflow behaves consistently across multiple configurations.

7. Validating Mixed-Version Reads

The first objective of the benchmark was validating that both schema versions could still be queried together correctly. This validation is important because v1 does not contain the `device_type` column while v2 introduces the new field.

The reader aligns all datasets to a unified schema before concatenation and querying. The benchmark confirms that the mixed mode correctly combines both dataset versions. For every dataset size, the number of loaded rows in mixed mode is approximately twice the value observed for v1 or v2 individually. This confirms that both dataset versions are successfully loaded together before query execution.

8. Query Runtime Analysis

After validating correctness, the next objective was evaluating the runtime overhead introduced by schema evolution. Figure 3 compares query runtime across all benchmark configurations.

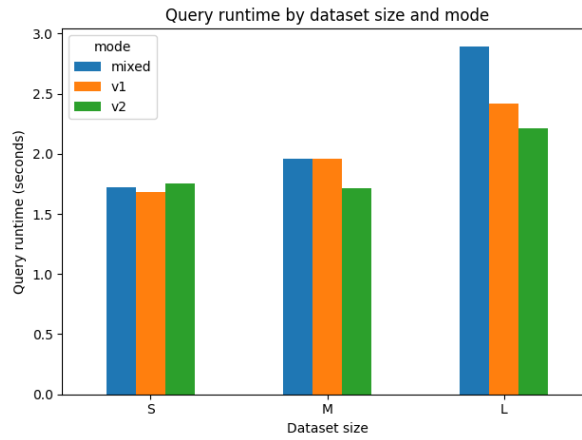


Figure 3. Query runtime by dataset size and reading mode.

The benchmark follows the expected trend. For all dataset sizes, the mixed mode produces the highest query runtime because the reader must load both dataset versions, align schemas, concatenate both tables, and finally execute the analytical query.

The additional overhead becomes more visible for the largest dataset configuration (L), where the mixed mode clearly requires more processing than reading a single schema version.

The results therefore confirm that schema evolution introduces additional processing costs while preserving backward compatibility.

9. Object Storage Behavior

The benchmark also validates that the reader interacts with the expected objects during execution. The object count remains fully coherent across all benchmark configurations. v1 reads one object, v2 also reads one object, while mixed reads both objects together. The total stored size also roughly doubles in mixed mode because both dataset versions are accessed simultaneously. This behavior confirms that the workflow correctly handles multiple dataset versions inside object storage.

10. Transfer Performance

The benchmark also measures upload and download throughput between the local environment and MinIO. Figure 4 presents the measured download throughput across all benchmark configurations.

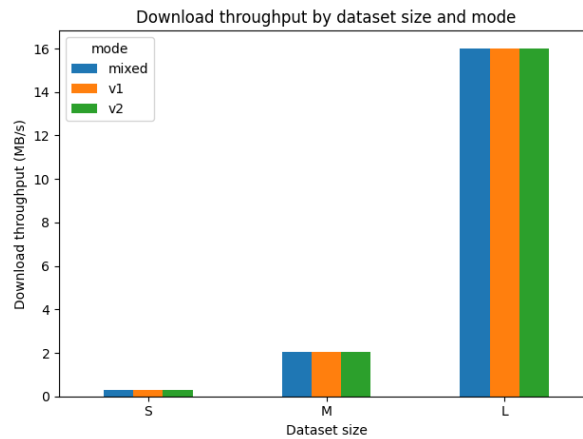


Figure 4. Download throughput by dataset size and mode

The throughput values scale consistently with dataset size. At the same time, differences between reading modes remain limited, which suggests that the transfer pipeline behaves similarly for v1, v2, and mixed. Because the benchmark was executed locally using Docker and MinIO, these values should be interpreted as relative measurements rather than production cloud performance.

11. Listing Time

The benchmark also measures object listing time during execution.

Figure 5 presents the measured listing times across all benchmark configurations.

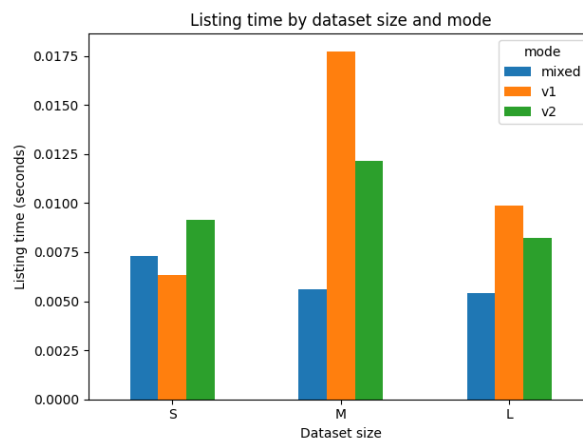


Figure 5. Listing time by d

The measured listing times remain very small throughout the benchmark, with all values staying below 0.02 seconds. Some variations appear between dataset sizes and reading modes, particularly for the medium (M) configuration where the v1 mode produces the highest listing time. However, these differences remain relatively small overall and do not significantly affect the execution workflow.

The results therefore suggest that object listing operations introduce only a negligible overhead compared to the full query execution process. This behavior remains consistent with the relatively small number of objects manipulated during the benchmark.

12.Limitations

Several limitations remain in the current implementation.

First, the benchmark was executed locally using Docker and MinIO. The measured performance therefore does not represent production cloud environments.

Second, only one benchmark execution was performed for each configuration. Running multiple iterations and averaging the results would improve measurement reliability.

Finally, the schema evolution scenario remains intentionally simple and only introduces one nullable column (`device_type`). More complex schema changes would require additional compatibility logic.

13.Conclusion

This project demonstrates a complete schema evolution workflow on S3-compatible object storage.

The implementation successfully combines versioned Parquet datasets, schema contracts, MinIO object storage, backward-compatible reads, and automated benchmark execution.

The benchmark confirms that mixed version reads remain possible after schema evolution while maintaining compatibility between dataset versions.

The experiment also highlights the main trade-off of the approach: mixed version reads improve compatibility, but they also introduce additional processing overhead, especially for larger datasets.

Overall, the workflow behaved consistently across all tested dataset sizes and successfully validated the schema evolution scenario implemented during the project.